CallFlow 미니 버전 구조 정리 및 향후 개선 계획

1. 문서 목적

이 문서는 현재 구현된 **LangGraph 기반 멀티 에이전트 상담 시스템의 구조**를 정리하고, 앞으로 어떤 방식으로 구조를 개선할지 상세히 설명하기 위한 문서이다.

현재 프로젝트는 최종 목표인 **전화 상담 AI 플랫폼 전체**를 한 번에 구현하는 것이 아니라, 그 안의 핵심인 **CallFlow 코어 엔진**을 먼저 만드는 단계에 있다. 따라서 이 문서는 다음 두 가지를 중심으로 정리한다.

- 현재까지 구현된 구조와 각 파일/모듈의 역할
- 앞으로의 변경 방향과 단계별 확장 계획

추가로, 이후 **Gemma 4 모델을 일부 노드에 도입하여 토큰 사용량과 성능을 비교하는 실험 계획**도 포함한다.

---

2. 현재 프로젝트의 전체 방향

현재 프로젝트는 다음 구조를 중심으로 동작한다.

- **Frontend**: React 기반 채팅 UI
- **Backend API**: FastAPI `/chat`
- **Orchestration**: LangGraph 상태 기반 그래프
- **Knowledge Search**: ChromaDB + 임베딩 기반 RAG
- **Specialist Flow**: FAQ 응답, callback 접수, vision fallback

즉, 지금 단계의 프로젝트는 다음과 같이 이해할 수 있다.

> **전화/음성 없이도 동작하는 CallFlow 코어 엔진의 미니 버전**

이 방향은 최종 기획서의 Tier 1(정보 상담), Tier 2(간단 액션)과 자연스럽게 연결된다.

---

3. 현재까지 구현된 구조 상세 정리

3.1 전체 디렉토리 관점

현재 프로젝트는 크게 다음 영역으로 나뉜다.

- `backend/` → FastAPI, LangGraph, RAG, Tool Layer
- `frontend/` → React 기반 대화형 데모 UI

지금 단계에서 핵심은 `backend/`이며, `frontend/`는 이 엔진을 테스트하기 위한 채팅 인터페이스 역할을 한다.

3.2 백엔드 구조 상세

`backend/main.py`

역할: - FastAPI 서버 실행 엔트리포인트 - `/chat` API 제공 - 프론트에서 전달한 대화 상태(session state)를 받아 LangGraph 실행 - 실행 결과를 다시 프론트로 반환

현재 특징: - 프론트에서 `collected_name`, `collected_phone`, `active_flow` 같은 상태를 같이 보냄 - 서버가 세션 저장소를 직접 들고 있는 구조는 아님 - 즉, 세션 유지의 중심은 아직 프론트 쪽에 가까움

현재 단계에서는 빠르게 실험하기에 적합하지만, 나중에 실제 전화/음성 세션으로 확장할 때는 **서버 중심 세션 구조**로 바뀌어야 한다.

`backend/app/graph.py`

역할: - LangGraph 그래프 정의 - 각 노드 등록 - `next_action` 값을 바탕으로 노드 분기 구성

현재 흐름 개요: 1. `intent_router` 2. `dialogue_manager` 3. specialist node 분기 - `faq_node` - `callback_node` - `vision_node` - `response_node` 4. 종료

즉 현재 구조는 다음과 같다.

```
사용자 입력
→ Intent 분류
→ 다음 행동 결정
→ specialist node 실행
→ response 생성
```

이 구조는 이미 LangGraph 기반 멀티 노드 구조이며, 최종 기획에서 말하는 **LangGraph 오케스트레이터의 축소판**으로 볼 수 있다.

`backend/app/state.py`

역할: - 그래프 전체에서 공유할 상태 정의

현재 주요 상태 필드: - `user_input` - `intent` - `next_action` - `collected_name` - `collected_phone` - `tool_result` - `final_response` - `active_flow`

장점: - callback 멀티턴 흐름을 관리하기 충분함 - FAQ / callback / vision fallback 정도의 미니 버전을 구현하기에는 적절함

한계: - retrieval, evidence, action 결과를 각각 분리해서 담기엔 부족함 - tenant, session_id, guardrail 등 미래 확장에 필요한 상태가 없음

즉 현재 state는 **MVP용 상태 구조**이고, 앞으로는 **엔진용 상태 구조**로 확장되어야 한다.

3.3 노드(Node) 구조 상세

`backend/app/nodes/intent_router.py`

역할: - 사용자의 발화를 보고 intent 분류 - 현재 분류 대상 예시: - `faq` - `callback` - `vision_needed` - `unsupported`

현재 특징: - `active_flow == callback`이면 다시 intent를 새로 분류하지 않고 callback 흐름 유지 - callback처럼 이미 시작된 멀티턴 흐름이 끊기지 않도록 설계됨

평가: - 지금 구조에서 매우 중요한 노드 - 이후 Gemma 4와 비교 실험을 가장 먼저 해보기 좋은 후보

`backend/app/nodes/dialogue_manager.py`

역할: - 현재 state를 보고 다음 행동(`next_action`) 결정 - callback 흐름이면 이름/전화번호 수집 단계 판단

현재 가능한 action 예시: - `route_faq` - `ask_name` - `ask_phone` - `route_callback` - `route_vision` - `route_unsupported`

현재 성격: - 자유형 agent라기보다는 **규칙 기반 오케스트레이터** - 상태를 보고 가장 안전한 다음 행동을 결정하는 컨트롤러 역할

평가: - 지금 프로젝트는 전화 상담 코어 엔진 미니 버전이므로, 이런 식의 **명시적 상태 제어**가 오히려 맞다 - 추후에도 Dialogue Manager는 자유형 ReAct agent보다는 **상태 기반 컨트롤러**로 유지하는 것이 유리하다

`backend/app/nodes/faq_node.py`

역할: - FAQ/RAG 응답 담당 specialist node - 사용자 질문을 받아 `faq_tool()` 또는 검색 도구를 호출 - 검색 결과를 `tool_result`에 저장

현재 한계: - 아직 retrieval/evidence/response로 역할이 분리되지 않음 - 검색 결과가 사실상 문자열 하나에 가까움 - 문서 근거의 출처, 페이지, 유사도, 복수 chunk 조합 정보가 부족함

평가: - 현재 FAQ specialist 역할은 동작하지만, 앞으로 가장 많이 구조가 바뀔 노드 - 최종적으로는 `retrieval_node` 성격으로 확장되어야 함

`backend/app/nodes/callback_node.py`

역할: - 이름/전화번호를 수집한 뒤 callback 처리 실행 - callback tool 호출 후 `tool_result` 저장

현재 장점: - 이미 Tier 2 액션의 최소 형태가 구현되어 있음 - 추후 `business_action_node` 또는 `action_node`로 일반화하기 좋은 구조

평가: - callback 기능은 지금 미니 버전에서 매우 중요한 데모 포인트 - 향후 callback 외에도 메모 저장, 요약 저장, 알림 등으로 확장 가능

`backend/app/nodes/response_node.py`

역할: - state를 바탕으로 최종 사용자 응답 생성 - `intent`, `next_action`, `tool_result` 등을 활용

현재 성격: - 문자열 기반 `tool_result`를 자연어 응답으로 바꾸는 후처리기에 가까움

현재 한계: - retrieval 결과가 구조화되면 이 노드도 같이 바뀌어야 함 - evidence 기반 응답 생성, 문서 부재 안내, 추가 질문 유도 등 세밀한 제어가 아직 부족함

평가: - 현재는 최종 응답 생성 노드로 충분하지만, - 앞으로는 **근거 기반 응답 생성기**로 업그레이드되어야 함

3.4 Tool Layer 현재 구조

`backend/app/tools/faq_tool.py`

역할: - ChromaDB에서 similarity search 실행 - 현재는 질문과 가장 비슷한 chunk를 찾아 문자열로 반환

현재 한계: - `file_name`, `page`, `score`, `chunk_index` 같은 metadata가 충분히 드러나지 않음 - 여러 chunk를 묶어서 reasoning하기 어려움 - evidence check를 위한 구조화된 결과가 부족함

특히 MakeShop 매뉴얼처럼 절차형 문서는 단일 top-1 검색보다: - 관련 chunk 여러 개 검색 - 처리 단계 묶음 구성 - 근거 강도 판정

이 훨씬 중요하다.

`backend/app/tools/callback_tool.py`

역할: - 콜백 요청 처리

현재는 callback 등록에 초점이 맞춰져 있으며, 향후 아래 액션으로 확장 가능한 출발점이 된다.

- callback 저장
- 상담 메모 저장
- 상담 요약 저장
- 알림 발송

`backend/app/tools/vision_tool.py`

역할: - 이미지 요청이 필요한 경우의 fallback 처리

현재는 실제 이미지 인식 기능이 아니라, "이미지가 필요하다"는 시스템 동작을 표현하는 mock 단계에 가까움

3.5 RAG 구조 현재 상태

`backend/app/rag/build_vector_db.py`

현재까지는 txt FAQ 중심 ingestion에서 시작했고, 이제는 PDF 기반 ingestion으로 확장하는 방향이 맞다.

현재/예상 데이터 위치: - `backend/data/faq_docs/faq.txt` - `backend/data/chroma/` - 이후 `backend/data/knowledge_docs/*.pdf`

MakeShop 매뉴얼은 단순 FAQ보다 프로세스 문서에 가깝기 때문에, 향후 retrieval/evidence 구조를 검증하기에 적합하다.

3.6 프론트엔드 구조 현재 상태

프론트는 React + TypeScript 기반으로, 현재는 다음 역할을 중심으로 한다.

- 메시지 리스트 표시
- 입력창 제공
- session state 유지
- callback flow 관련 상태 전달
- 백엔드 `/chat` 호출

즉 지금 프론트는 **관리자 플랫폼**이 아니라, CallFlow 코어 엔진을 테스트하기 위한 **대화형 데모 UI**이다.

이건 현재 단계에 적절하며, 최종 기획의 Flow Designer/관리자 플랫폼은 이후 별도 레이어로 붙이는 것이 맞다.

---

4. 현재 구조의 강점

4.1 LangGraph가 중심이라는 점

이건 가장 큰 장점이다. 지금부터 LangGraph 기반으로 상태와 노드를 분리해두면, 나중에 음성 파이프라인, WebSocket, Tool Layer, 가드레일까지 자연스럽게 확장할 수 있다.

4.2 역할 분리가 이미 시작되어 있다는 점

현재도 다음 분리가 존재한다.

- Intent 분류
- 다음 행동 결정
- FAQ specialist
- callback specialist
- response 생성

완전하지는 않지만, 이미 멀티 노드 구조가 잡혀 있어 다음 단계로 가기 쉽다.

4.3 callback 멀티턴 흐름이 구현되어 있다는 점

이건 이후 slot filling, 인증 흐름, 제품 식별 흐름 등으로 확장하기 좋은 기반이다.

4.4 프론트와 상태 연동이 구현되어 있다는 점

세션 구조가 아직 완벽하진 않지만, 대화 흐름을 빠르게 실험하고 디버깅하는 데 유리하다.

---

5. 현재 구조의 한계

5.1 Retrieval / Evidence / Response 분리가 아직 없음

현재 faq 검색 결과는 거의 문자열 1개 수준이다. 그 결과: - 문서에 없는 내용인지 판단하기 어려움 - 복수 근거를 묶어 설명하기 어려움 - 근거 부족 시 fallback 경로가 약함

5.2 Node와 Tool의 책임이 충분히 분리되지 않음

현재는 일부 노드가 실행과 판단을 함께 수행하는 경향이 있다. 장기적으로는: - **Node = 판단, 흐름 제어** - **Tool = 실행**으로 분리해야 한다.

5.3 State가 미래 확장 대비로는 얇음

지금 state는 MVP용으로 충분하지만, 나중에는 다음 요소가 필요해진다.

- tenant_id
- session_id
- retrieval_result
- evidence_result
- action_result
- guardrail_flags

5.4 Action 계층이 callback 하나에 묶여 있음

현재 callback은 잘 구현된 편이지만, 향후 action node로 일반화되어야 한다.

예: - callback 저장 - 상담 메모 저장 - 상담 요약 저장 - 알림 발송

5.5 Response Node가 아직 근거 기반 응답 생성기로는 부족함

현재는 결과 문자열을 자연어로 다듬는 수준에 가깝다. 앞으로는: - 문서 근거 전달 - 문서 부재 응답 - 추가 질문 유도 - callback 연결

까지 제어할 수 있어야 한다.

---

6. 앞으로의 목표 구조

앞으로 미니 버전은 아래 구조를 목표로 바뀌어야 한다.

```
FastAPI /chat
→ LangGraph Orchestrator
    → Intent Router
    → Dialogue Manager
    → Retrieval Node
    → Evidence Check Node
    → Response Node
    → Business Action Node
```

그리고 실행 계층은 Tool Layer로 분리한다.

```
knowledge_search_tool
collect_info_tool
register_callback_tool
vision_request_tool
```

핵심 설계 원칙: - **노드 = 판단/흐름 제어** - **툴 = 실행** - **상태(state) = 노드 간 정보 공유**

---

7. 단계별 변경 계획

Step 1. PDF ingestion 구조로 전환

대상: - `build_vector_db.py`

변경 목표: - txt FAQ 위주 ingestion에서 PDF 문서 ingestion으로 확장 - MakeShop PDF 기준으로 문서 chunking - metadata 저장 강화 - `file_name` - `page` - `chunk_index` - `file_type` - `doc_type`

왜 중요한가: - MakeShop 문서는 상태 전이/절차형 문서이므로 페이지/청크 기반 근거 관리가 중요함

Step 2. `faq_tool`을 `knowledge_search_tool`로 확장

현재: - 질문 → 문자열 반환

변경 후: - 질문 → 구조화된 dict 반환

예상 구조: - `query` - `retrieved_count` - `enough_evidence` - `results[]` - `content` - `score` - `file_name` - `page` - `chunk_index`

효과: - retrieval 결과를 evidence check와 response에서 재사용 가능 - 근거 중심 답변 구조로 확장 가능

Step 3. `faq_node`를 retrieval node 성격으로 확장

현재: - 검색 결과 문자열 저장

변경 후: - 구조화된 retrieval 결과를 state에 저장 - preview 문자열은 하위 호환용으로만 유지

효과: - `faq_node`가 실제로는 `retrieval_node` 역할을 하게 됨 - 다음 단계에서 evidence check를 붙이기 쉬워짐

Step 4. `evidence_check_node` 추가

역할: - retrieval 결과가 충분한지 판단 - 복수 chunk를 함께 볼 필요가 있는지 판단 - 문서에 없는 경우 fallback 플래그 설정 - 애매할 경우 추가 질문 또는 안전 응답 유도

특히 MakeShop 매뉴얼처럼 절차형 문서에서는, 단순 top-1 검색보다 이런 evidence 판정 단계가 훨씬 중요하다.

예: - 주문 취소 - 환불 처리 - 교환 처리 - 상태 변경 흐름

Step 5. `response_node`를 근거 기반 응답 생성기로 업그레이드

현재: - 문자열 결과를 자연어 답변으로 바꿈

변경 후: - evidence 결과를 바탕으로: - 근거 기반 응답 - 문서 부재 응답 - 추가 질문 유도 응답 - callback 유도 응답

즉 response node는 단순 후처리기가 아니라, **근거 전달 책임이 있는 응답 생성 노드**가 된다.

Step 6. callback을 일반 business action으로 확장

현재: - callback only

변경 후: - `business_action_tool` 개념으로 일반화

예상 확장: - `register_callback` - `save_note` - `save_summary` - `send_alert`

callback은 그중 하나의 액션이 된다.

Step 7. state 구조 확장

향후 권장 필드 예시: - `session_id` - `tenant_id` - `intent` - `active_flow` - `slots` - `retrieval_result` - `evidence_result` - `action_result` - `final_response` - `guardrail_flags`

이렇게 하면: - 음성 세션 - 테넌트별 문서 분리 - 본인인증 - 이미지 기반 기능

등으로의 확장이 쉬워진다.

---

8. 앞으로 권장하는 폴더 구조

현재 구조를 크게 흔들지 않으면서, 역할 기반으로 정리하면 다음과 같은 형태가 좋다.

```
backend/app/
  config.py
  graph.py
  state.py

  nodes/
    intent_router.py
    dialogue_manager.py
    retrieval_node.py
    evidence_check_node.py
    response_node.py
    business_action_node.py
    vision_node.py

  tools/
    knowledge_search_tool.py
    callback_tool.py
    business_action_tool.py
    vision_tool.py

  rag/
    build_vector_db.py
    loaders.py
    splitters.py

  prompts/
    intent_router_prompt.py
    response_prompt.py
    evidence_check_prompt.py
```

핵심은 도메인 이름보다 역할 이름으로 바꾸는 것이다.

예: - `faq_node` → `retrieval_node` - `callback_node` → `business_action_node` 일부

---

9. Gemma 4 도입 계획

향후에는 Gemma 4 모델을 사용해, **토큰 사용량을 줄이면서 기존 모델 대비 성능을 비교하는 실험**을 진행할 계획이다.

중요한 점은: - 처음부터 모든 노드에 Gemma 4를 넣는 것이 목표가 아님 - 어느 노드에 넣었을 때 비용/성능 효율이 가장 좋은지 확인하는 것이 목표임

9.1 Gemma 4를 먼저 넣어볼 후보 노드

후보 1. Intent Router

이유: - 짧은 입력 - 분류형 작업 - 출력 구조가 단순 - 토큰 절감 효과가 크기 쉬움

가장 먼저 비교 실험하기 좋은 노드

후보 2. Evidence Check Node

이유: - retrieval 결과 요약 판단이 중심 - 비교적 구조화된 입력 - 분류/판단 성격이 강함

Intent Router 다음 후보로 적절함

후보 3. Response Node

이유: - 사용자 체감 품질에 가장 직접적 영향을 줌 - 말투, 자연스러움, 안전 응답이 중요함

따라서 가장 마지막에 비교하는 것이 좋다.

9.2 비교 실험 구조

Baseline

- 기존 OpenAI 기반 현재 구조

Variant A

- Intent Router만 Gemma 4

Variant B

- Intent Router + Evidence Check를 Gemma 4

Variant C

- Intent Router + Evidence Check + Response까지 Gemma 4

이렇게 점진적으로 비교하면, 어디까지 Gemma 4로 대체해도 품질이 유지되는지 확인 가능하다.

9.3 비교 지표

1. 품질

평가 질문셋 예시: - 단순 FAQ - 절차형 질문 - 다중 조건 질문 - 문서에 없는 질문 - callback 유도 질문

예: - "주문 취소는 어떻게 처리하나요?" - "환불은 어떤 순서로 진행되나요?" - "교환과 환불 차이가 뭐예요?" - "이 문서에 없는 기능인데 가능한가요?" - "담당자 연결이 안 되면 콜백 요청 가능한가요?"

---

2. 토큰 사용량

기록할 항목: - input tokens - output tokens - total tokens - 평균 turn당 token 사용량 - node별 token 사용량

특히 분리해서 봐야 할 노드: - intent router - evidence check - response node

---

3. 응답 속도

기록할 항목: - intent router latency - retrieval latency - evidence latency - response latency - 전체 turn latency

---

9.4 실험의 목표

실험 목표는 단순한 모델 교체가 아니다.

진짜 목표는 다음과 같다.

> **어떤 노드를 Gemma 4로 바꾸는 것이 가장 효율적인지 찾는 것**

즉, - 토큰 사용량 감소 - 응답 시간 유지 또는 개선 - 품질 저하 최소화

이 세 가지를 함께 만족하는 지점을 찾는 것이 중요하다.

---

10. 결론

현재 프로젝트는 전체 전화 상담 플랫폼을 완성하는 단계가 아니라, 그 안의 핵심인 **CallFlow 코어 엔진**을 만드는 단계이다.

지금까지 구현된 구조는 이미 다음 장점을 갖고 있다.

- LangGraph 중심 구조
- Intent / Dialogue / Specialist / Response 분리
- callback 멀티턴 흐름 구현
- RAG 연동 시작

하지만 앞으로는 아래 방향으로 더 정교하게 바뀌어야 한다.

- PDF 기반 ingestion 강화
- `knowledge_search_tool` 구조화
- retrieval / evidence / response 분리
- callback을 business action으로 일반화
- state 확장
- node/tool 책임 분리

그리고 이후 Gemma 4를 일부 노드에 단계적으로 적용하여, 토큰 사용량과 품질을 함께 비교하는 실험까지 이어질 예정이다.

한 문장으로 정리하면 다음과 같다.

> **지금은 전화 상담 서비스 전체가 아니라, 향후 음성·인증·이미지 기능까지 붙일 수 있는 CallFlow 코어 엔진을 먼저 완성하는 단계이다.**